

Use XMPP to Create Your Own Google Talk Client



Instant communication is now the essence of social networking and the Internet. The popular Google Talk, which uses XMPP (Extensible Messaging and Presence Protocol), made this Instant Messaging protocol prominent among open standards protocols. Exploring XMPP (formerly known as the Jabber protocol) is fun—it is a transparent and simple architecture. Once you understand it, you can easily write your own XMPP/GTalk clients from the ground up, using the friendly and powerful Python scripting language.

XMPP is an XML-based open standards protocol. It uses XML streams to implement the entire message communication system.

XMPP is used in a de-centralised client-server architecture, in which a server acts as an intermediary for the message transfer, and also manages services like the user account registration, authentication, buddy list database, etc. Since our primary focus is on clients, we won't dig any more into the server part—we will just consider the server to be a 'black box' entity available at a specific IP address/hostname and port number, which meaningfully responds to our XMPP requests.

We connect to the server using a TCP socket in our program. An explanation of networking, TCP/IP, IP addresses, ports and sockets is too much information to put into this article, so if these concepts are new to you, you could visit http://en.wikipedia.org/wiki/Internet_socket for

some quick reading.

Since our purpose in this article is to code our own Google Talk client, the hostname of the server we will use is gmail.com, and the port we will use is 5222, the default port for the XMPP service.

The sequence of the initial interaction between client and server is as follows:

1. The client connects to the server and sends credentials like the username and password.
2. The server validates the received credentials against its user database and sends a response to the client.
3. When authentication is successful, the client receives a response containing *presence notification* data. This is a collection of *presence* data from different buddies of the user, which the client authenticated to the server. Presence is explained below.

As already noted, the communication between client and server is in the form of